

作业 3: AI for 软件工程

探讨 AI 在软件全生命周期中的应用可能性，并举例说明（含实验）。

1.1 内容选择

我选择探讨的是第二个主题 AI 辅助的代码伦理审查（使用 AI 代码审查工具（如 SonarQube、DeepCode）扫描学生实验代码，分析潜在漏洞（如内存泄漏、逻辑错误），关联技术缺陷可能引发的社会风险（如金融系统崩溃）。伦理讨论：结合 AI 审查结果，开展“技术缺陷是否等同于道德失范”的课堂辩论，强化责任意识）。

2.1 程序设计与潜在问题

针对要探讨的这个问题，我先自己写了一个 c++ 程序，其中包含了几个潜在的问题，具体代码如下：

```
#include <bits/stdc++.h>
using namespace std;
struct Student
{
    char name[50];
    int age;
    char student_id[10];
};
int main()
{
    char temp_name[100];
    cout << "请输入学生姓名（使用不安全的 gets）： ";
    gets(temp_name);
    Student student1;
    strcpy(student1.name, temp_name);
    cout << "请输入学生年龄： ";
    cin >> student1.age;
    cin.ignore();
    char log_message[100];
    char *dynamic_id = new char[20];
    strcpy(dynamic_id, "SN1234567");
    sprintf(log_message, "学生信息： %s， 年龄： %d", student1.name,
student1.age);
```

```
cout << "\n 学生信息 (存在风险版本)" << endl;
cout << "姓名: " << student1.name << endl;
cout << "年龄: " << student1.age << endl;
cout << "日志: " << log_message << endl;
delete[] dynamic_id;

return 0;
}
```

不难发现，strcpy()、gets()等函数在使用的时候都存在着栈缓冲区溢出的潜在问题，下面将用 chatgpt 分析潜在的漏洞关联并让它生成技术缺陷可能引发的社会风险。

3.1 使用 Chatgpt 得到结果

一开始输入的提示词如下：



得到的 gpt 反馈如下：

代码行号	风险类型	潜在后果（技术 + 社会/伦理）	关联案例
15 (gets(temp_name))	不安全输入 — gets 导致栈缓冲区溢出（无边界读入）	用户可输入超长数据导致栈溢出，引起内存破坏、程序崩溃或任意代码执行；在真实系统中可能被远程利用而造成服务中断、数据泄露或被植入恶意逻辑，带来隐私/安全与法律责任。	Morris 蠕虫（1988）：利用 fingerd 中用 gets 等不安全读入的缓冲区溢出作为感染载体，在互联网上迅速传播并导致大规模服务中断。 CMU计算... +1
17 (strcpy(student1.name, temp_name))	无界拷贝 — strcpy 导致缓冲区溢出/结构体邻接内存被覆盖	将大于 student1.name（50 字节）的数据复制入会覆盖相邻内存（如 age、student_id 或返回地址），导致数据篡改、崩溃或代码执行；在金融/基建等关键系统，这类缺陷可被利用引发欺诈、服务中断、甚至系统被接管，造成重大经济与社会影响。	Code Red（2001）：利用微软 IIS 的缓冲区溢出（属于无界写入/缓冲区溢出类漏洞）实现远程代码执行并大规模感染企业服务器，造成严重的业务中断与安全事件。 维基百科 +1
24 (sprintf(log_message, "学生信息: %s, 年龄: %d", student1.name, student1.age))	无界格式化写入与日志注入 (sprintf 无长度限制)	当 student1.name 很长或包含特殊内容时，sprintf 可能溢出 log_message（100 字节），导致内存破坏；另外把未经清洗的用户输入写入日志，可能引发日志注入/伪造（影响审计）、信息泄露或在审计/追责场景下造成误导；在金融或医疗系统会导致错误审计结论、合规/隐私问题，甚至触发连锁故障。	金融领域的自动化/防护不足案例（类比）：Knight Capital（2012）——因部署/软件防护与校验不足导致短时间内数百万笔错误交易、公司损失数亿美元，强调“缺乏防御性编程与检验/回退逻辑”在真实世界的严重后果（与未做边界检查/日志与校验不足属同类系统工程失误）。 Henri Codrillg +1

概括来说，存在以下漏洞：

gets()函数：使用 string + getline(cin, str) 或 fgets/gets_s 函数来替换并且限制最大长度与检查返回值。

strcpy()使用安全拷贝（如 strncpy/strncpy 并确保 NUL 结尾）或使用 string 来避免手工管理缓冲区；为结构内的定长数组设定清晰上界并在拷贝前验证长度。

sprintf()改用 snprintf 并传入目标缓冲区大小或使用 ostream/string 来构造日志消息；对写入日志的用户输入应做清洗/转义以防日志注入；为关键路径加防护逻辑（长度检查、异常处理、失败回退）。

也可以看到，chatgpt 针对这几潜在的问题，延申出了其在技术与社会中的潜在后果，并结合了具体的历史案例进行分析，比如蠕虫病毒利用 gets()函数的不安全输入进行传播感染；Knight Capital 公司由于缺乏防护与回退措施而损失很多。

4.1 安全代码生成

接下来，我让 chatgpt 生成了一个较为安全的程序（源程序的修改版本），具体的程序代码如下：

```
#include <bits/stdc++.h>
using namespace std;
// 使用 string 替代固定长度字符数组，自动管理内存并防止越界写入
struct Student
{
    string name;          // 比较安全，不会因 strcpy 而溢出
    int age = 0;          // 初始化，避免未定义值
    string student_id;    // 使用 string 来保存学号
};
int main()
```

```

{
    // 使用 string + getline 安全读取整行输入（不会写出缓冲区边界）
    string temp_name;
    cout << "请输入学生姓名: ";
    getline(cin, temp_name);
    // 如果希望对姓名长度做限制，可以显式截断并记录：
    const size_t MAX_NAME_LEN = 50; // 原代码 name 缓冲区为 50
    if (temp_name.size() > MAX_NAME_LEN - 1)
    {
        // 截断以保证兼容旧有定长存储限制（如果需要的话），并告知用户
        temp_name = temp_name.substr(0, MAX_NAME_LEN - 1);
        cout << "警告：姓名过长，已被截断到 " << (MAX_NAME_LEN - 1) << "
个字符。\\n";
    }
    Student student1;
    student1.name = temp_name; // 安全赋值，无溢出
    // 读取年龄：使用 getline 然后验证，避免直接用 >> 导致输入残留或格式错误
    cout << "请输入学生年龄: ";
    string age_line;
    getline(cin, age_line);
    // 验证年龄为正整数且在合理范围内
    try
    {
        // 使用 stoi 解析并检查范围
        long parsed = stol(age_line);
        if (parsed < 0 || parsed > 150)
        {
            cerr << "无效的年龄范围，设置为 0。\\n";
            student1.age = 0;
        }
        else
            student1.age = static_cast<int>(parsed);
    }
    catch (const exception &e)
    {
        // 解析失败时使用默认值并告知
        cerr << "年龄输入解析失败，已设置为 0。\\n";
        student1.age = 0;
    }
    // 学号示例：使用 string 而不是 new char[]，避免手动管理内存
    string dynamic_id = "SN1234567"; // 更安全，也更简洁
    // 如果我们需要强制学号长度（原来 student_id 是 10 字节），可以截断或校验：
    const size_t MAX_ID_LEN = 10;
    if (dynamic_id.size() > MAX_ID_LEN - 1)

```

```

        dynamic_id = dynamic_id.substr(0, MAX_ID_LEN - 1);
        student1.student_id = dynamic_id;
        // 构建日志信息: 使用 ostringstream 或 snprintf 的安全版本, 避免 sprintf
        导致溢出
        ostringstream oss;
        oss << "学生信息: " << student1.name << ", 年龄: " << student1.age;
        string log_message = oss.str(); // 安全的字符串构造
        cout << "\n--- 学生信息 (修正版) ---" << endl;
        cout << "姓名: " << student1.name << endl;
        cout << "年龄: " << student1.age << endl;
        cout << "学号: " << student1.student_id << endl;
        cout << "日志: " << log_message << endl;
        // 不需要显式 delete, 所有对象在离开作用域时自动清理 (RAII)
        return 0;
    }

```

可以看到, 大部分之前提到的潜在的问题都得到了修改。

5.1 结论以及思考

在软件工程实验中使用 AI 代码审查工具来扫描我们的代码是一个非常有价值的做法。以前我写代码, 更多关注的是程序能不能跑、能不能通过测试的问题, 在一些比赛中甚至会为了效率高而不经意间引入一些潜在的内存相关的问题。但使用 AI 来审查能指出一些我们平时容易忽略的问题, 比如内存泄漏、不安全函数使用、逻辑漏洞等, 体现出了“能跑”并不等于“写得好”。

更重要的是, 当把这些问题放到真实系统的背景下去看时, 会发现它们并不是小事。程序中的一个数组越界、内存错误, 可能只是程序崩溃; 但如果出现在金融系统中, 可能导致交易数据出错; 出现在医疗系统中, 甚至可能影响病人的生命安全。也就是说, 代码里的技术缺陷或者一个小漏洞, 在真实的生活中中有可能直接转化为社会风险的。

在此基础上, 作业中提出“技术缺陷是否等同于道德失范”这个问题, 我觉得非常有讨论意义。我的看法是: 技术缺陷本身不一定等于道德问题。人不是万能的, 写代码写出漏洞是一件很正常的事情, 没有人能够检查出自己程序中的所有漏洞, 甚至 AI 也不能保证给出的修改后版本就一定万无一失, 不能因为一个错误就直接给开发者贴上“不負責任”的标签。

但问题在于, 当风险已经被明确指出之后, 开发者是否选择认真对待。如果 AI 已经提示某段代码存在严重漏洞, 而开发者为了赶进度、图省事, 选择忽略甚至掩盖这些问题, 那么这就不再只是技术能力问题, 而是责任意识的问题了。在这种情况下, 技术缺陷就和道德问题产生了联系。

以我自己的视角来说, 我认为 AI 代码审查工具起到的是“提醒和约束”的作用。它

让问题变得更透明，也让“没发现问题”这个理由变得站不住脚。对于开发者来说，看到明确的风险提示却不去修复，本身就是一种不负责任的行为。

——23121769 赵文宇

2025.12.18